# 桌面端项目面试 Q&A

基于 HIS 药房模块（WPF + WinForms）实战整理

---

## 一、MVVM 架构

### Q1：为什么选择 WPF + MVVM?

**参考回答：**

1. **表现层分离**：XAML 负责 UI，ViewModel 负责逻辑，数据绑定自动同步
2. **便于测试**：ViewModel 无 UI 依赖，可单元测试
3. **团队协作**：、美工负责 XAML，后端负责 ViewModel，并行开发
4. **社区成熟**：CommunityToolkit.Mvvm 提供完整基础设施

**项目中实践**：用 CommunityToolkit.Mvvm 重构原 WinForms 模块，代码可维护性提升约 50%

---

### Q2：MVVM 中 ViewModel 之间如何通信?

**常用方式：**

1. **Messenger（消息机制）**：通过 WeakReferenceMessenger 发送和接收消息
2. **依赖注入**：构造函数注入其他 ViewModel 或服务
3. **共享服务**：单例服务存放共享数据

```
// 发送方
WeakReferenceMessenger.Default.Send(new RefreshInventoryMessage());

// 接收方
WeakReferenceMessenger.Default.Register<RefreshInventoryMessage>(this, (r, m) =>
{
    LoadInventory();
});
```

**项目中实践**：处方发药完成后，通过 Messenger 通知库存管理界面刷新数据

---

### Q3：WPF 数据绑定不生效怎么排查?

**排查步骤：**

1. 确认 `DataContext` 是否正确设置
2. 检查绑定路径是否正确（大小写敏感）
3. 确认对象是否实现了 `INotifyPropertyChanged`
4. 查看 Visual Studio 输出窗口的绑定错误
5. 检查 `UpdateSourceTrigger` 模式

```
<!-- 正确绑定示例 -->
<TextBlock Text="{Binding PatientName}" />
<Button Command="{Binding ConfirmCommand}" />
```

---

## 二、串口通信

### Q4：HIS 项目中串口通信的应用场景？

**医院常见设备：**

1. **自动发药机**：处方配药完成后，发送指令弹出对应药盒
2. **煎药机**：接收煎药处方，监控煎药进度
3. **电子秤**：中药饮片称重
4. **条码枪**：药品扫码

**协议类型**：RS232 / RS485（需电平转换）

---

### Q5：串口通信如何实现？

**核心代码：**

```
public class SerialPortHelper : IDisposable
{
    private SerialPort _serialPort;

    public void Open(string portName, int baudRate)
    {
        _serialPort = new SerialPort(portName, baudRate, Parity.None, 8, StopBits.One);
        _serialPort.DataReceived += OnDataReceived;
        _serialPort.Open();
    }

    private void OnDataReceived(object sender, SerialDataReceivedEventArgs e)
    {
        // 在独立线程处理接收数据
        var buffer = new byte[_serialPort.BytesToRead];
        _serialPort.Read(buffer, 0, buffer.Length);
        ParseProtocol(buffer);
    }

    public void Send(byte[] command)
    {
        _serialPort?.Write(command, 0, command.Length);
    }

    public void Dispose()
    {
        _serialPort?.Close();
        _serialPort?.Dispose();
    }
}
```

**注意事项：**

- 医院设备协议通常是自定义二进制协议，需按协议文档解析
- 串口是独占资源，需做好异常处理与断开重连
- 使用单独线程处理串口通信，避免阻塞 UI

---

### Q6：TCP Socket 在设备监控中的应用？

**场景**：实时监控发药机、煎药机状态

```
public class DeviceMonitor
{
    private TcpClient _client;
    private NetworkStream _stream;

    public async Task ConnectAsync(string ip, int port)
    {
        _client = new TcpClient();
        await _client.ConnectAsync(ip, port);
        _stream = _client.GetStream();
        _ = ReceiveLoopAsync();
    }

    private async Task ReceiveLoopAsync()
    {
        var buffer = new byte[1024];
        while (_client.Connected)
        {
            var bytesRead = await _stream.ReadAsync(buffer);
            if (bytesRead > 0)
            {
                ParseDeviceStatus(buffer.Take(bytesRead).ToArray());
            }
        }
    }

    public event Action<DeviceStatus> OnStatusChanged;
}
```

---

## 三、性能优化

### Q7：大数据量查询如何避免 UI 卡顿？

**常用方案：**

1. **后台线程查询**：`Task.Run` 在后台线程执行数据库查询
2. **分页加载**：使用 VirtualizingStackPanel 虚拟化
3. **异步回调 UI**：`Dispatcher.Invoke` 或 `await` 回 UI 线程更新

```
private async Task LoadDataAsync()
{
    IsLoading = true;
    var data = await Task.Run(() => _service.GetLargeDataset());
    Dispatcher.Invoke(() =>
    {
        Items = new ObservableCollection<Item>(data);
        IsLoading = false;
    });
}
```

**项目中实践**：跨月库存报表等大数据量查询不阻塞主界面

---

### Q8：本地缓存如何设计？

**场景**：医院内网环境，字典数据（科室、药品基础信息）频繁访问

```
public class LocalCacheService
{
    private static readonly Dictionary<string, (object Value, DateTime Expires)>
_cache = new();
    private static readonly TimeSpan DefaultExpiration = TimeSpan.FromMinutes(30);

    public T GetOrSet<T>(string key, Func<T> factory, TimeSpan? expiration = null)
    {
        if (_cache.TryGetValue(key, out var entry) && entry.Expires > DateTime.Now)
        {
            return (T)entry.Value;
        }

        var value = factory();
        _cache[key] = (value, DateTime.Now + (expiration ?? DefaultExpiration));
        return value;
    }
}
```

---

### Q9：频繁 Invoke 导致 UI 卡顿如何优化？

**问题**：高频数据更新时，每次都 Invoke 会导致消息队列堆积

**优化方案**：

1. **节流**：限制更新频率，如每 100ms 更新一次
2. **批量更新**：收集多次更新，合并为一次 UI 刷新
3. **只保留最新数据**：更新前判断是否需要刷新

```
// 节流示例
private DateTime _lastUpdate = DateTime.MinValue;
private readonly TimeSpan _throttleInterval = TimeSpan.FromMilliseconds(100);

public void UpdateData(Data data)
{
    if (DateTime.Now - _lastUpdate < _throttleInterval) return;
    _lastUpdate = DateTime.Now;
    Dispatcher.Invoke(() => CurrentData = data);
}
```

---

## 四、依赖属性与命令

### Q10：依赖属性（Dependency Property）的作用？

**参考回答**：

支持 WPF 核心特性：

- **数据绑定**：依赖属性天然支持绑定

- **样式与模板**：可通过 Trigger、Style 设置
- **动画**：支持动画变更
- **默认值继承**：可在父元素上设置，子元素继承

```
public static readonly DependencyProperty IsBusyProperty =
    DependencyProperty.Register(nameof(IsBusy), typeof(bool), typeof(MyControl),
        new PropertyMetadata(false));

public bool IsBusy
{
    get => (bool)GetValue(IsBusyProperty);
    set => SetValue(IsBusyProperty, value);
}
```

---

### Q11：WPF 命令（Command）与 Click 事件的区别?

| 特性 | Command | Click 事件 |
|---|---|---|
| 解耦 | UI 与逻辑完全分离 | 需在 Code-Behind 写逻辑 |
| 可测试 | ViewModel 层可测试 | 依赖 UI 元素 |
| 绑定 | 支持数据绑定 | 不支持 |
| 复用 | 多个按钮可绑定同一命令 | 需要方法调用 |

---

## 五、集合变更通知

### Q12：ObservableCollection 修改必须在 UI 线程?

**关键点：**

- `INotifyPropertyChanged`：属性变更通知，WPF 会自动封送到 UI 线程
- `ObservableCollection.CollectionChanged`：集合变更通知，WPF **不会**自动封送

**解决方案**：使用 `BindingOperations.EnableCollectionSynchronization`

```
// 跨线程更新集合
private readonly object _lock = new();
private ObservableCollection<Item> _items = new();

public MainViewModel()
{
    BindingOperations.EnableCollectionSynchronization(_items, _lock);
}

public void AddItem(Item item)
{
    lock (_lock)
    {
        _items.Add(item);
    }
}
```

---

## 六、项目亮点提炼

### Q13：药房模块最有技术含量的点是什么？

**参考回答（建议）：**

1. **设备协议对接**：串口协议解析、多品牌设备兼容、异常断连重连
2. **业务流程复杂**：处方从医生站传入 → 配药 → 发药确认，涉及多个系统交互
3. **性能要求高**：药房高峰期并发大，需保证 UI 响应速度
4. **数据一致性**：药品库存、处方、费用多系统同步

**核心成果**：

- 处方发药流程自动化，减少人工操作
- 药品批次与效期跟踪，FIFO 出库策略减少过期损耗
- 多设备统一管理框架，支持快速接入新设备

---

### Q14：从 WinForms 迁移到 WPF 遇到了哪些挑战？

**主要挑战：**

1. **观念转变**：从事件驱动到数据绑定
2. **性能考量**：WPF 渲染开销更大，需注意可视化树复杂度
3. **第三方控件**：WinForms 的 DevExpress 与 WPF 控件用法不同
4. **原有代码复用**：部分业务逻辑需抽象到服务层

**迁移策略**：模块化迁移、原业务逻辑下沉到服务、逐步替换 UI

---

## 七、反问准备

### Q15：桌面端岗位可能问的问题

1. 客户端更新机制怎么做？（ClickOnce、Squirrel）
2. 如何处理离线和在线场景？
3. 有没有做过客户端性能优化？（启动速度、内存占用）
4. 对 WPF 性能的优化经验？（XAML 优化、可视化树）

---

建议：重点准备串口通信、MVVM 通信机制、性能优化三个方向，确保能说出"为什么这样做"